School of Computing
CA326 Year 3 Project Proposal Form

SECTION A

Project Title: FootStat

Student 1 Name: _Jaime de Vivero Woods_ ID Number: _19447494_

Student 2 Name: _Alen Joy_ ID Number: _18313576_

Staff Member Consulted for supervision: _Marija Bezbradica_

Project Description (2 pages max)

Introduction

Describe the general area / problem covered by the project.

For our third-year project, our idea is to develop an Android app which takes publicly available statistics on major European soccer leagues from sports websites and displays them in various formats and layouts, allowing things like comparing teams and viewing results and performance in past games. We also plan to implement a ranking system based on various algorithms to help determine the relative strength of various teams and players.

Outline

Outline the proposed project.

The app will take statistics on major European soccer leagues from websites in JSON format. The JSON data is interpreted and used to rank teams and players using an ELO rating system. The ELO ratings for players are based on the level of the competition on which teams complete their first scored match. After each game played, the rating of players and teams will be updated according to their result and taking into account the relative playing strength of the opponent. We will also use a Glicko RD rating system to estimate the accuracy of ELO results. Additionally, users will have the access to features standard on most sports statistics applications, for example, viewing the scores from previous games, viewing upcoming match information, player information (e.g. goals, passes, cards etc.) and match tables. Information is planned to be viewable in various formats, for example, a user may select a certain league and filter out all the players that participated in it in order of descending goals.

```
class GameApp {
}
"gameid": "soccer.g.2247118"
                    "start_time": "Sat, 06 Nov 2021 12:30:00 +0000",
                    "is_time_tba": false,
                    "season_phase_id": "season.phase.season",
                    "game_type": "Regular Season",
                    "winning_team_id": "soccer.t.89",
                    "outcome_type": "outcome.type.won",
                    "home_team_id": "soccer.t.90",
                    "away_team_id": "soccer.t.89",
                    "week_number": "11",
                    "navigation_links": {
                        "boxscore": {
                            "url": "\/soccer\/premier-league\/manchester-united-manchester-city-2247118\/"
                        },
                        "match_page": {
                            "url": "\/soccer\/premier-league\/manchester-united-manchester-city-2247118\/"
                        },
                        "league_home": {
                            "url": "\/soccer\/premier-league\/"
                        },
                        "league_scores": {
                            "url": "\/soccer\/premier-league\/scoreboard\/"
                        }
                    },
                    "status_display_name": "Finished",
                    "status_description": "Final",
                    "status_type": "status.type.final",
                    "total_away_points": "2",
                    "current_period_id": "2",
                    "total_home_points": "0",
                    "game_time_elapsed": "95",
                    "game_time_elapsed_display": "90'+5'",
                    "season": "2021",
                    "subleague": "soccer.l.fbgb",
                    "subleague_display_name": "Premier League",
        "group_position": "5",
```


In this JSON data there is information about the team's position in the Premier League, the final score of the match, the current week of the season and the points the team got from the match.

Background

Where did the ideas come from?

We initially got the idea from Dr Mark Roantree's project ideas webpage. The sports app idea seemed interesting to us because we follow sports, especially the European soccer leagues. After trying some pre-existing apps (like LiveScore and Yahoo Sports), we tried to think of ways they can be improved and more statistics we would like to see that were not on those applications.

Goals

What will the project aim to achieve?

The goal is to create an android app that can track various statistics for European football leagues, have the ability to contrast and compare teams or potentially players, and rank teams and players using the ELO rating and Glicko RD algorithms mentioned previously.

Why/when/where/how will it be useful?

We hope this will be useful to people looking for an application that does standard statistics retrieval combined with the ability to make comparisons between teams, and also algorithmic ranking. There are few apps on the market that do all of these things combined and the few that do require a payment or subscription in order to have access to all the data. These stats can be useful for people who place bets on the winners of matches, competitions and individual trophies.

Programming language(s) and tools

List the proposed language(s) you plan to use and any other software tools you might use (compiler, database, web server, etc.)

Languages: Java, Kotlin, SQL, JavaScript

Tools: MySQL Workbench, Adobe XD, IntelliJ Idea, Android Studio, Webstorm, Socket.io, Retrofit API, Websockets, Git

We will most likely use Java and Kotlin for the majority of development of the app itself. We plan to use IntelliJ Idea and Android Studio for this. We will request the JSON data from the website containing the statistics with a REST API using Retrofit. We will use SQLite databases on Android Studio to store the JSON data and send it to the client in various client-side views and formats. The calculations for the ELO and Glicko algorithms will also take place on device if we go ahead with SQLite. This mitigates the requirement for a dedicated web server for storing databases. For designing and prototyping the UI, we will use Adobe XD in combination with Android Studio's design tools.

Our Java/Kotlin code will be tested using Junit 5 and debugging tools integrated into Idea and Android Studio. Features will be tested as much as possible on separate git branches before being merged to the master branch. SQLite code will be tested using the following test harnesses: TCL tests, TH3 tests, SQL logic tests. These will validate the basic integrity and logic of our databases. Further tests, speedtest1 and mptester, will be used to measure the speed and performance of database access and usage.

Breakdown of work

Clearly identify who will undertake which parts of the project.
It must be clear from the explanation of this breakdown of work both that each student is responsible for separate, clearly-defined tasks, and that those responsibilities substantially cover all of the work required for the project.

Student 1

Student 1 should complete this section.

Student 1 will develop the Java/Kotlin code with the Retrofit API in Android Studio to get the JSON data from the website containing the statistics to be interpreted. The SQLite database which will store the JSON data to be interpreted will be developed by both Student 1 and Student 2 in Android Studio. After a design for the code for Java/Kotlin is made and the class diagrams are developed detailing the functionality of the code, Student 1 and Student 2 will develop the code together, splitting an even number of classes for each student to do.

Student 2

Student 2 should complete this section.

Student 2 will primarily focus on UI design and prototyping using Adobe XD and design tools built into Android Studio. We will divide the Java/Kotlin code and work on the classes together. The SQLite / other database will be developed by both students. We plan to use the "code with me" function in Studio and Idea to collaboratively write the code for Kotlin/Java. We will also split up the task of doing research on implementation ideas between us, for any feature we would like to add. Git will be used with branches for both students onto which individual features will be implemented and then tested before being merged onto the master branch.